# ANALYTICAL SOLVING PROBLEM

## 1. Water Jug Problem

Two jugs of **5 litres** and **3 litres** are available. Neither jug has measuring marks.

**Task:**
Using the available operations (Fill, Empty, Pour), obtain exactly **4 litres** in the 5-litre jug. Show all intermediate states.

**SOLUTION:**

**Given**

- Jug A = 5 litres
- Jug B = 3 litres
- Goal: Obtain exactly 4 litres in the 5-litre jug.

**Allowed Operations**

- Fill a jug completely.
- Empty a jug.
- Pour water from one jug to another until one jug is full or the other is empty.

**State Representation**

Each state is represented as (5L Jug, 3L Jug).

Initial State: (0, 0)

Goal State: (4, 0)

**Intermediate Steps**

| Step | Operation | State (5L, 3L) |
|---|---|---|
| 1 | Initial State | (0,0) |
| 2 | Fill 5L jug | (5,0) |
| 3 | Pour 5L → 3L | (2,3) |
| 4 | Empty 3L jug | (2,0) |
| 5 | Pour 5L → 3L | (0,2) |
| 6 | Fill 5L jug | (5,2) |
| 7 | Pour 5L → 3L until 3L is full | (4,3) |
| 8 | Empty 3L jug | (4,0) ✓ Goal Reached |

**State Space Diagram**

(0,0)

│ Fill 5L

▼

(5,0)

│ Pour to 3L

▼

(2,3)

│ Empty 3L

▼

(2,0)

│ Pour to 3L

▼

(0,2)

│ Fill 5L

▼

(5,2)

│ Pour to 3L

▼

(4,3)

│ Empty 3L

▼

(4,0) ✓ Goal

**CONCLUSION:**

The required 4 litres is obtained in the 5-litre jug.

Final State: (4,0)

**2. River Crossing Problem**

A farmer needs to transport a Wolf, Goat, and Cabbage across a river.

**Rules:**

- Boat carries only the farmer and one item.
- Wolf cannot stay alone with the Goat.
- Goat cannot stay alone with the Cabbage.

**Task:**
Solve the problem using State Space Search and show all intermediate states.

**SOLUTION:**

**Given**

A farmer must transport a Wolf (W), Goat (G), and Cabbage (C) across a river.

**Rules**

- The boat can carry only the farmer and one item.
- The Wolf cannot be left alone with the Goat.
- The Goat cannot be left alone with the Cabbage.

**State Representation**

- L = Left Bank
- R = Right Bank

Each state is represented as:

(Farmer, Wolf, Goat, Cabbage)

Initial State: (L, L, L, L)

Goal State: (R, R, R, R)

**Intermediate States**

| Step | Action | State (Farmer, Wolf, Goat, Cabbage) |
|---|---|---|
| 1 | Initial State | (L, L, L, L) |
| 2 | Take Goat to Right | (R, L, R, L) |
| 3 | Farmer returns alone | (L, L, R, L) |
| 4 | Take Wolf to Right | (R, R, R, L) |
| 5 | Bring Goat back | (L, R, L, L) |
| 6 | Take Cabbage to Right | (R, R, L, R) |
| 7 | Farmer returns alone | (L, R, L, R) |
| 8 | Take Goat to Right | (R, R, R, R) ✓ Goal Reached |

**State Space Diagram**

(L,L,L,L)

│ Take Goat

▼

(R,L,R,L)

│ Farmer returns

▼

(L,L,R,L)

│ Take Wolf

▼

(R,R,R,L)

│ Bring Goat back

▼

(L,R,L,L)

│ Take Cabbage

▼

(R,R,L,R)

│ Farmer returns

▼

(L,R,L,R)

│ Take Goat

▼

(R,R,R,R) ✓ Goal

**CONCLUSION:**

The farmer successfully transports all three items across the river without violating any rule.

Final State: (R, R, R, R) ✓ All items safely reach the right bank.

**3. Online Cab Booking System**

Customers choose Mini, Sedan, SUV, or Prime.

**Task:**
Identify the AI agent used and write the pseudocode.

**SOLUTION:**

AI Problem-Solving Agent

Agent Used: Utility-Based Agent

**Reason:**

A Utility-Based Agent selects the best cab option based on:

- Passenger preference
- Cab availability
- Travel cost
- Comfort
- Estimated arrival time

It chooses the option that provides the highest utility (best overall service) for the customer.

**Pseudocode**

```
START

Input pickup location

Input destination

Input preferred cab type

Check available cabs

IF preferred cab is available THEN

    Assign the cab

ELSE

    Suggest another available cab

ENDIF

Calculate fare

Confirm booking

Display booking details

STOP
```

**Working**

1. Customer enters pickup and destination locations.
2. The system checks the availability of the selected cab type.

3. If available, the cab is assigned.
4. Otherwise, the system suggests another available cab.
5. The fare is calculated, and the booking is confirmed.

**CONCLUSION:**

**AI Agent Used:** Utility-Based Agent

**Reason:** It selects the most suitable cab by considering availability, cost, comfort, and passenger preference, providing the best travel option to the customer.

**4. Monkey and Banana Problem**

A monkey wants to reach bananas hanging from the ceiling.

**Task:**
Represent the problem using **State Space Search** and show all intermediate states.

**SOLUTION:**

**Given**

A monkey wants to get bananas hanging from the ceiling. A box is available in the room.

Actions

- Walk
- Push the box
- Climb the box
- Grab the bananas

**State Representation**

Each state is represented as:

(Monkey Position, Box Position, Monkey Status, Banana Status)

- Monkey Status: On Floor / On Box
- Banana Status: Not Reached / Reached

Initial State:
(Door, Window, On Floor, Not Reached)

Goal State:
(Center, Center, On Box, Reached)

**Intermediate States**

| Step | Action | State (Monkey Position, Box Position, Monkey Status, Banana Status) |
|---|---|---|
| 1 | Initial State | (Door, Window, On Floor, Not Reached) |
| 2 | Monkey walks to the box | (Window, Window, On Floor, Not Reached) |
| 3 | Monkey pushes the box to the center | (Center, Center, On Floor, Not Reached) |
| 4 | Monkey climbs onto the box | (Center, Center, On Box, Not Reached) |
| 5 | Monkey grabs the bananas | (Center, Center, On Box, Reached) ✓ Goal |

**State Space Diagram**

(Door, Window, On Floor, Not Reached)

│ Walk

▼

(Window, Window, On Floor, Not Reached)

│ Push Box

▼

(Center, Center, On Floor, Not Reached)

│ Climb Box

▼

(Center, Center, On Box, Not Reached)

│ Grab Banana

▼

(Center, Center, On Box, Reached) ✓ Goal

**CONCLUSION:**

The monkey reaches the bananas by following these actions:

1. Walk to the box.
2. Push the box below the bananas.
3. Climb onto the box.
4. Grab the bananas.

Goal State: (Center, Center, On Box, Reached)

**5. Weighted Graph Problem**

Find the shortest path from **S** to **G** using **Uniform Cost Search (UCS)**.

SOLUTION:

**Given Graph**

```
     A
   /  \
  2      3
 /        \
S          B
 \        /
  4      2
   \    /
     G
```

**Edge Costs**

- S → A = 2
- S → G = 4
- A → B = 3
- B → G = 2

**Step 1: Initial Priority Queue**

| Priority Queue | Cost |
|---|---|
| S | 0 |

**Step 2: Expand Node S**

From **S**, two paths are available:

- S → A = 2
- S → G = 4

**Priority Queue**

| Node | Cost |
|---|---|
| A | 2 |

| Node | Cost |
|---|---|
| G | 4 |

**Step 3: Expand Node A**

From **A**:

- A → B = 3

Total Cost = **2 + 3 = 5**

**Priority Queue**

| Node | Cost |
|---|---|
| G | 4 |
| B | 5 |

**Step 4: Expand Node G**

Goal node **G** is selected because it has the **lowest cost (4)**.

**Search stops.**

**Node Expansion Order**

**S → A → G**

**Optimal Path**

S ⟶ G

**Path Cost**

| Path | Total Cost |
|---|---|
| S → G | **4** |
| S → A → B → G | **7** |

**CONCLUSION:**

- **Algorithm Used:** Uniform Cost Search (UCS)

- **Optimal Path: S → G**
- **Total Cost: 4**

Since UCS always expands the node with the **lowest cumulative cost**, it selects the direct path **S → G**, which is the shortest and least-cost path.